

UNIVERSITÀ DEGLI STUDI DI SALERNO
Corso di Laurea in Ingegneria Informatica



Anno Accademico 2025/2026

Relazione tecnica

StudyPlanner

TRACCIA PROGETTUALE 2

Gruppo 16

Autori

Mario Di Gregorio
Gabriele Alfano
Angelo Ambrosio

Matricole

0612708906
0612710143
0612709750

Indice

1	Executive Summary	3
1.1	Introduzione	3
2	Descrizione dell'app	3
2.1	Obiettivo dell'applicazione	3
2.2	Tipologia di utenti	3
2.3	Problema affrontato	3
2.4	Scenari d'uso principali	3
3	Requisiti	4
3.1	Requisiti implementati	4
3.2	Funzionalità considerate ma non implementate	6
3.3	Feature avanzate scelte	6
3.4	Limitazioni note	6
4	Progettazione dell'app	7
4.1	Struttura generale	7
4.2	Architettura logica	7
4.3	Flusso di navigazione	8
4.4	Wireframe delle schermate	8
4.5	Modello dati e persistenza locale	12
4.6	Schermate principali	12
4.6.1	Home	13
4.6.2	Corsi	13
4.6.3	Esami e scadenze	13
4.6.4	Calendario	14
4.6.5	Obiettivi	14
4.6.6	Pomodoro	15
4.7	Layout responsive e adattamento mobile	15
4.8	Iconografia	16
5	Scelte tecnologiche	16
5.1	Framework utilizzato	16
5.2	Librerie principali	17
5.3	Modalità di gestione della persistenza	17
6	Implementazione	17
6.1	Organizzazione del codice	17
6.2	Gestione dello stato	18
6.3	Gestione della navigazione	18
6.4	Passaggio dati tra schermate	19
6.5	Gestione dei dati persistenti	19
6.6	Componenti e widget principali	20
6.7	Parti significative o complesse	20
6.7.1	Calendario mensile	20
6.7.2	Suggerimenti automatici	21
6.7.3	Pomodoro	21

7 Istruzioni per l'esecuzione	21
7.1 Prerequisiti	21
7.2 Comandi di avvio	22
8 Conclusioni	22

1 Executive Summary

1.1 Introduzione

StudyPlanner è un'applicazione Flutter pensata per supportare gli studenti universitari nell'organizzazione dello studio, nella gestione dei corsi, nel monitoraggio di esami e scadenze, nella pianificazione delle sessioni e nel controllo dei propri obiettivi.

2 Descrizione dell'app

2.1 Obiettivo dell'applicazione

L'obiettivo di StudyPlanner è fornire allo studente universitario un ambiente unico per organizzare il proprio percorso di studio. L'app raccoglie in modo integrato le informazioni relative a corsi, esami, scadenze, sessioni di studio e obiettivi personali, permettendo all'utente di avere una visione ordinata delle attività da svolgere e dei progressi raggiunti.

La soluzione punta quindi a semplificare la pianificazione quotidiana e il monitoraggio dello studio, affiancando alle funzionalità principali, la presenza di strumenti di supporto come riepiloghi, calendario, suggerimenti automatici e timer Pomodoro.

2.2 Tipologia di utenti

L'app è rivolta principalmente a studenti universitari che devono coordinare più insegnamenti, scadenze e periodi di studio.

studenti che vogliono organizzare il semestre in modo ordinato; studenti che preparano esami con più date e priorità diverse; studenti che preferiscono visualizzare lo studio in forma di calendario; studenti che desiderano alternare studio concentrato e pause con il metodo Pomodoro.

2.3 Problema affrontato

Il problema principale è la frammentazione delle informazioni. Uno studente spesso usa strumenti diversi per calendario, lista di cose da fare, appunti sui corsi, promemoria degli esami e monitoraggio delle ore di studio.

StudyPlanner riduce questa dispersione e frammentazione delle informazioni, collegando ogni elemento al corso di riferimento e rendendo disponibili i dati principali in una dashboard unica.

L'app permette quindi di passare da una gestione casuale dello studio a una gestione più consapevole, visuale e misurabile.

2.4 Scenari d'uso principali

- **Inserimento dei corsi:** lo studente aggiunge gli insegnamenti del semestre con docente, CFU, stato, voto desiderato e note.

- **Gestione delle scadenze:** lo studente registra esami, appelli, consegne o progetti con data, priorità e stato.
- **Pianificazione dello studio:** lo studente apre il calendario, seleziona un giorno e crea sessioni di studio collegate a un corso.
- **Controllo obiettivi:** lo studente crea attività da completare, assegna priorità, tempi stimati e tempi effettivi.
- **Monitoraggio:** lo studente consulta la dashboard per vedere ore pianificate, ore svolte, scadenze imminenti e distribuzione dello studio.
- **Studio concentrato:** lo studente usa il Pomodoro integrato per alternare 25 minuti di studio e 5 minuti di pausa.

3 Requisiti

3.1 Requisiti implementati

I requisiti sono classificati in funzionali (**RF**) e non funzionali (**RNF**). Per ogni requisito sono indicati priorità e rischio tecnico.

ID	Area	Requisito	Priorità	Rischio Tecnico
RF-01	Corsi	Inserimento, visualizzazione, modifica ed eliminare dei corsi con nome, docente, semestre, CFU, stato, note e voto obiettivo.	1	Medio
RF-02	Esami e scadenze	Gestione degli esami, degli appelli, delle consegne e delle scadenze con titolo, corso associato, data, tipologia, priorità, stato, note e risultato.	1	Medio
RF-03	Pianificazione studio	Creazione sessioni di studio associate a un giorno specifico e a un corso, con durata pianificata, durata effettivamente svolta, tipologia e completamento.	1	Alto
RF-04	Attività e obiettivi	Gestione task e obiettivi con descrizione, corso associato, scadenza, priorità, stato, tempo stimato e tempo effettivo.	1	Medio
RF-05	Ricerca corsi	Ricerca corsi per nome e filtrare l'elenco in base allo stato del corso.	2	Basso

ID	Area	Requisito	Priorità	Rischio
RF-06	Filtri obiettivi	Filtrare gli obiettivi per stato, priorità e testo inserito dall'utente.	2	Basso
RF-07	Filtri calendario	Filtrare gli elementi del calendario distinguendo tra sessioni di studio, esami e obiettivi.	2	Medio
RF-08	Dashboard riepilogativa	Mostrare i corsi attivi, gli esami futuri, gli obiettivi aperti, le ore svolte, il progresso generale e le scadenze imminenti.	1	Medio
RF-09	Distribuzione studio	Mostrare la distribuzione del tempo di studio per corso tramite degli indicatori visuali.	2	Medio
RF-10	Calendario mensile	Visualizzare un calendario mensile con sessioni, esami e obiettivi collocati nei giorni corrispondenti.	1	Alto
RF-11	Pomodoro	Fornire un timer per alternare studio concentrato (25 minuti) e pausa breve(5 minuti).	2	Medio
RF-12	Suggerimenti automatici	Proporre sessioni di studio in base agli esami futuri entro una finestra temporale di 21 giorni.	2	Medio
RF-13	Persistenza locale	Salvare e ricaricare i dati locali tramite serializzazione JSON e shared_preferences .	1	Alto
RF-14	Navigazione responsive	Adattamento della navigazione dell'app mostrando una sidebar su schermi larghi e una bottom navigation su smartphone.	2	Medio
RNF-01	Usabilità	L'app deve essere chiara, coerente, con schermate comprensibili e interazioni semplici.	1	Medio
RNF-02	Responsività	L'interfaccia deve adattarsi correttamente a browser, emulatori e schermi mobili.	2	Medio
RNF-03	Manutenibilità	Il codice deve essere leggibile e organizzato logicamente, anche se concentrato principalmente nel file <code>main.dart</code> .	4	Basso

ID	Area	Requisito	Priorità	Rischio
RNF-04	Robustezza dei dati	Le modifiche effettuate dall'utente devono essere salvate e recuperate dopo il riavvio dell'applicazione.	2	Alto
RNF-05	Semplicità di installazione	Il progetto deve avere dipendenze limitate per facilitare l'esecuzione durante la discussione.	2	Basso

3.2 Funzionalità considerate ma non implementate

Alcune funzionalità sono state considerate ma non inserite nella versione ultima dell'applicazione:

- autenticazione utente e profili multiutente;
- sincronizzazione cloud tra dispositivi;
- notifiche push per promemoria automatici;
- importazione da Google Calendar o calendari universitari esterni;
- caricamento di materiali allegati ai corsi;
- gestione completa di sessioni universitarie reali, appelli multipli e prenotazioni ufficiali.

La scelta progettuale è stata concentrare il lavoro sulle funzionalità centrali richieste: CRUD, collegamento dei dati, navigazione, persistenza locale, pianificazione, statistiche e feature avanzate integrate.

3.3 Feature avanzate scelte

StudyPlanner implementa le seguenti feature:

1. **Calendario mensile con agenda giornaliera:** permette di visualizzare sessioni, esami e obiettivi direttamente sui giorni del mese.
2. **Timer Pomodoro:** alterna studio concentrato e pause brevi con timer 25/5.
3. **Suggerimenti automatici:** genera proposte di studio in base agli esami futuri entro una finestra temporale di 21 giorni.

3.4 Limitazioni note

Le principali limitazioni della versione attuale sono:

- I dati sono locali e non sincronizzati online;
- Non è presente autenticazione o registrazione utente;
- Il sistema gestisce le date in modo intuitivo, pur adottando un modello semplificato rispetto alla complessità di un calendario universitario reale;

- La versione mobile richiede particolare attenzione nella gestione delle schermate dense di elementi, in particolare nel calendario mensile.

4 Progettazione dell'app

4.1 Struttura generale

L'app è organizzata in sei sezioni principali, pensate per coprire le diverse aree della gestione dello studio universitario:

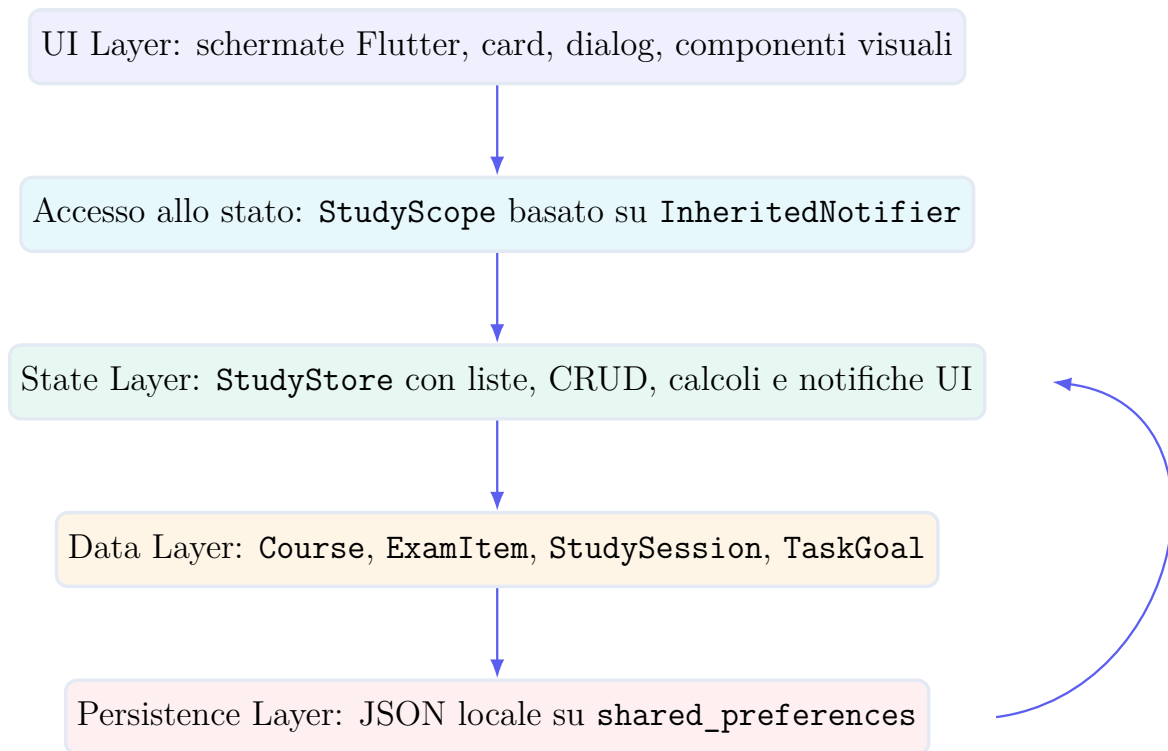
- **Home/Panoramica:** dedicata a riepiloghi, statistiche e scadenze imminenti;
- **Corsi:** dedicata alla gestione degli insegnamenti;
- **Esami:** dedicata ad appelli, consegne e scadenze;
- **Calendario:** dedicato alla pianificazione delle sessioni di studio;
- **Obiettivi:** dedicata alle attività e ai traguardi personali;
- **Pomodoro:** dedicata al timer di studio concentrato.

Questa suddivisione permette di mantenere separate le aree funzionali principali, pur collegandole attraverso dati condivisi e relazioni logiche tra corsi, esami, sessioni e obiettivi.

4.2 Architettura logica

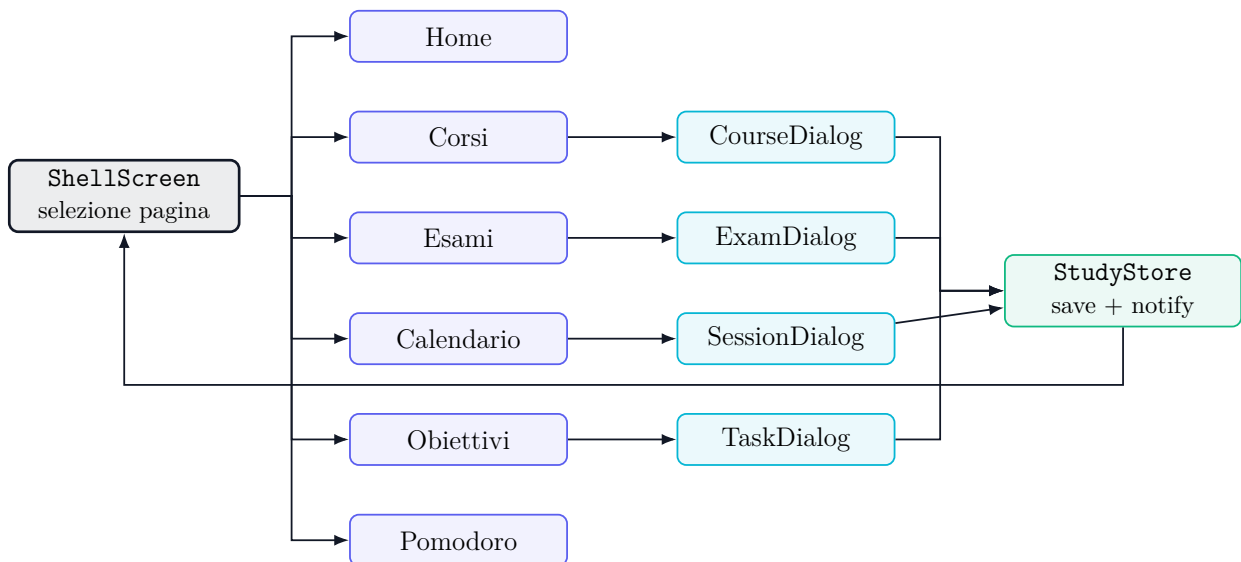
L'architettura interna segue una struttura semplice ma coerente:

- **UI Layer:** schermate, card, dialog e componenti visuali;
- **Accesso allo stato:** `StudyScope`, che permette alle schermate di recuperare lo stato condiviso;
- **State Layer:** `StudyStore`, basato su `ChangeNotifier`, con liste, operazioni CRUD, calcoli e notifiche alla UI;
- **Data Layer:** modelli Dart con metodi `toJson()` e `fromJson()`;
- **Persistence Layer:** salvataggio locale tramite JSON e `shared_preferences`.



4.3 Flusso di navigazione

La navigazione parte da `ShellScreen`, che controlla la sezione selezionata e mostra la schermata corrispondente. Le operazioni di creazione/modifica/eliminazione avvengono tramite dialog modali e aggiornano lo stato condiviso.

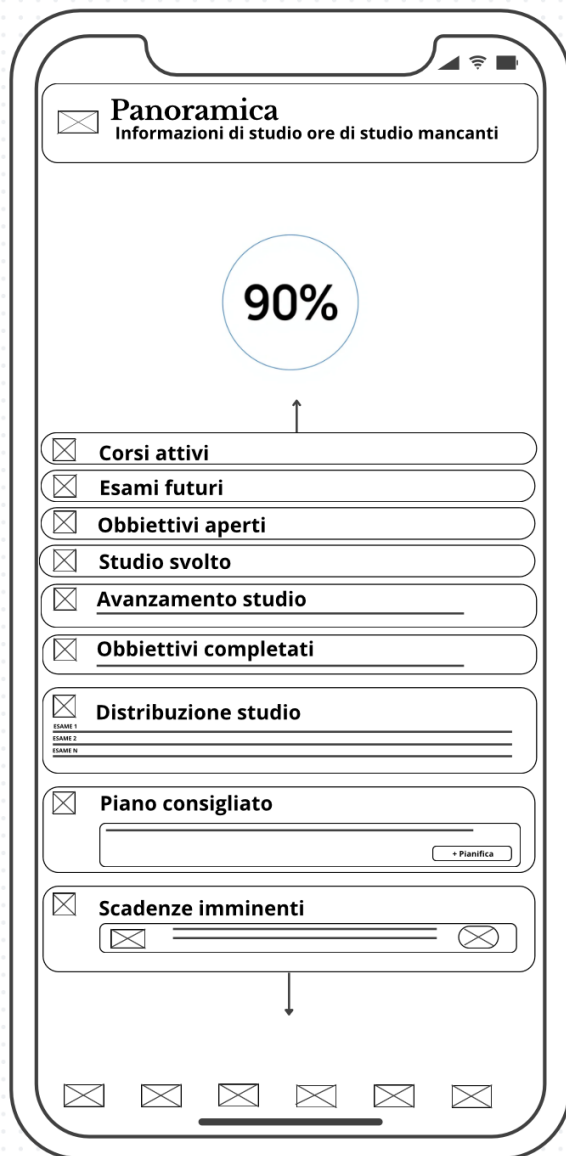


4.4 Wireframe delle schermate

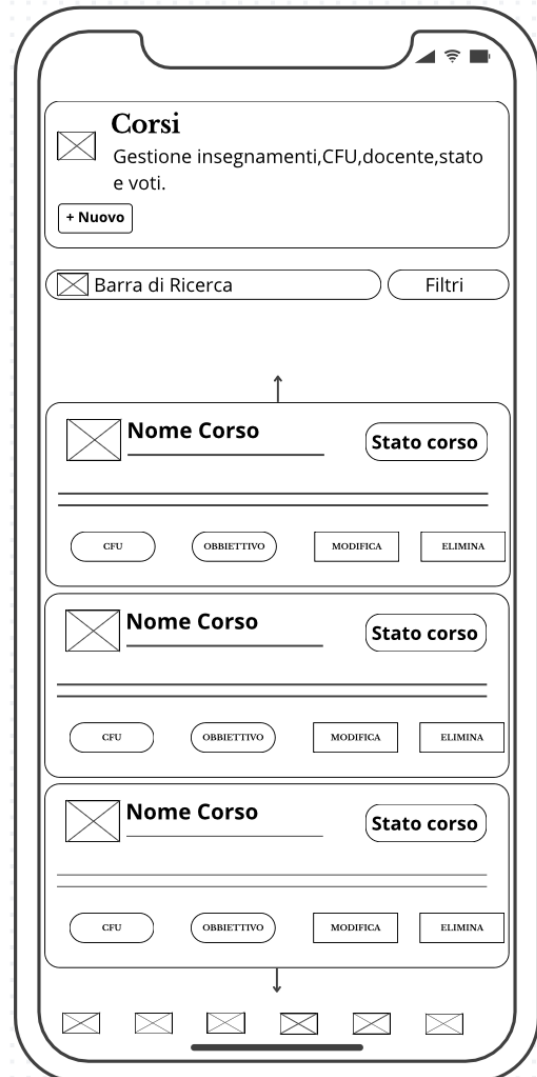
I wireframe sono stati inseriti come parte fondamentale della fase di Design dell'interfaccia. Essi mostrano una bozza strutturale generale delle schermate prima della realizzazione grafica definitiva, evidenziando la disposizione degli elementi principali, la gerarchia delle informazioni e le azioni disponibili per l'utente.

Le schermate sono in tutto sei: Home, Corsi, Esami/scadenze, Calendario, Obiettivi e Pomodoro. Questo ordine consente di collegare i wireframe al flusso di navigazione e alle schermate finali.

HOME



CORSI



ESAMI

CALENDARIO

Pianifica sessioni, esami, obiettivi direttamente sui giorni del mese

+Aggiungi

Mese - Anno

<

>

Lun - Mar - Mer - Gio - Ven - Sab - Dom

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28

● Sessioni studio

● Esami/cadenze

● Esami/cadenze

Agenda

AGGIUNGI

NUMERO ATTIVITÀ

TEMPO PER ATTIVITÀ

STATO ATTIVITÀ

Nome Attività

Stato corso

STATO ATTIVITÀ

MODIFICA

ELIMINA

Sintesi Mese

OBIETTIVI

☐

Obiettivi
Organizza attività, tempi, stimati e avanzamento dei corsi.

+ Nuovo

☐

OBIETTIVI

XX%

☐

Barra di Ricerca

Priorità

Stato

☐

Nome obiettivo

Priorità

TEMPO TOTALE PROGRAMMATO - TEMPO EFFETTIVAMENTE DEDICATO

STATO

MODIFICA

ELIMINA

☐

Nome obiettivo

Priorità

TEMPO TOTALE PROGRAMMATO - TEMPO EFFETTIVAMENTE DEDICATO

STATO

MODIFICA

ELIMINA

☐

☐

☐

☐

☐

☐

POMODORO

☐

Pomodoro
Timer per alternare studio concentrato e pause brevi

☐

Studio concentrato
25:00

☐Avvia

☐Reset

☐

☐

☐

☐

☐

☐

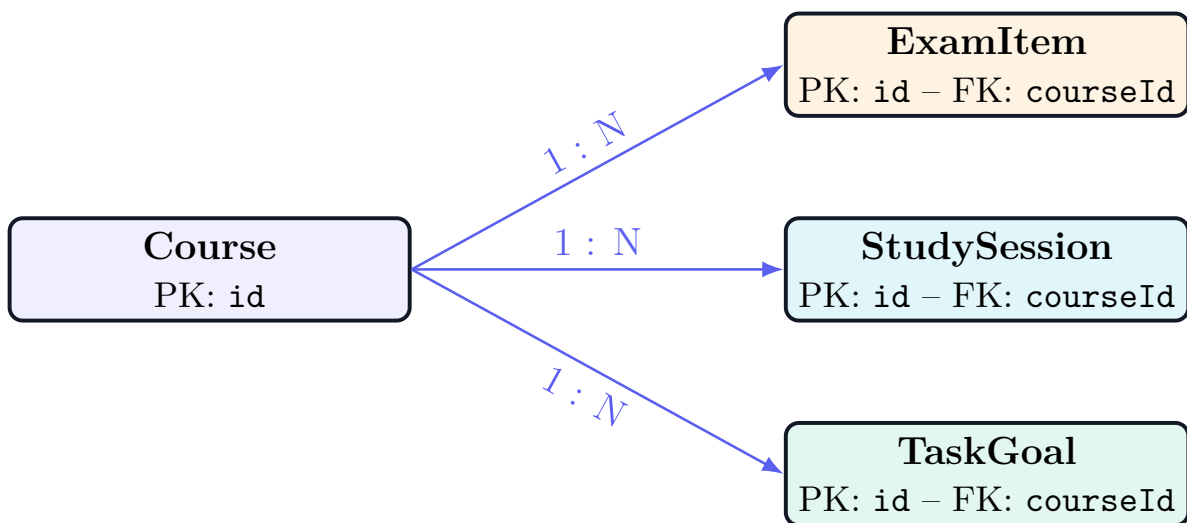
4.5 Modello dati e persistenza locale

Il modello dati dell'app è organizzato intorno a quattro entità principali: corsi, esami/-scadenze, sessioni di studio e obiettivi. Ogni elemento ha un identificativo `id`, usato dal codice per riconoscerlo in modo univoco.

Gli elementi collegati a un corso utilizzano l'attributo `courseId`. In questo modo un esame, una sessione di studio o un obiettivo possono essere associati al corso corretto. Non viene utilizzato un database relazionale, quindi questi collegamenti non sono gestiti da un DBMS, ma direttamente dal codice dell'app.

Un corso può quindi essere collegato a più esami, più sessioni di studio e più obiettivi; allo stesso tempo, un corso può esistere anche senza elementi associati.

Entità	Identificativo	Collegamenti	Attributi principali
Course	id	—	name, teacher, semester, cfu, status, desiredGrade, finalGrade, notes.
ExamItem	id	courseId → Course.id	title, date, type, priority, status, notes, result.
StudySession	id	courseId → Course.id	title, date, minutesPlanned, minutesDone, kind, completed.
TaskGoal	id	courseId → Course.id	title, description, dueDate, priority, status, estimatedMinutes, actualMinutes.



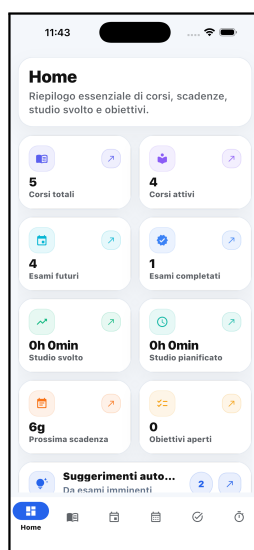
Quando un corso viene eliminato, il codice elimina anche gli esami, le sessioni e gli obiettivi collegati al suo `id`. In questo modo si evita che nell'app restino elementi non più associati ad alcun corso.

4.6 Schermate principali

Le schermate sono progettate per essere coerenti sia a livello grafico sia funzionale. Ogni schermata usa un'intestazione comune, card arrotondate, colori e componenti riutilizzabili.

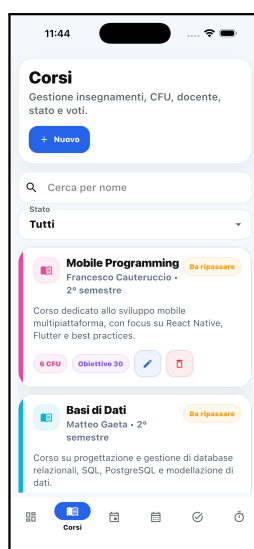
4.6.1 Home

La schermata Home raccoglie i dati più importanti: corsi attivi, esami futuri, obiettivi aperti, studio svolto, progresso generale, distribuzione dello studio per corso, scadenze imminenti e piano consigliato.



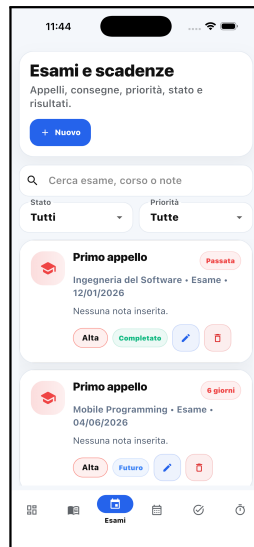
4.6.2 Corsi

La schermata Corsi permette di visualizzare, cercare, filtrare, aggiungere, modificare ed eliminare insegnamenti. Il dettaglio del corso è rappresentato direttamente nella card e nel dialog di modifica, evitando schermate intermedie non necessarie.



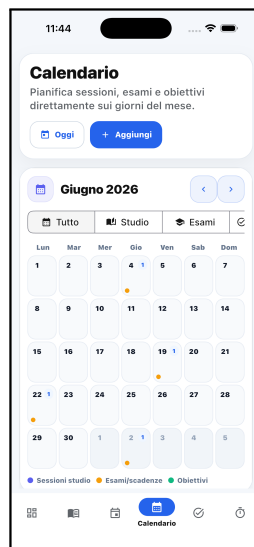
4.6.3 Esami e scadenze

La schermata Esami e scadenze contiene appelli, consegne e progetti. Ogni elemento mostra corso associato, data, tipologia, priorità, stato, note ed eventuale risultato.



4.6.4 Calendario

Nel Calendario è possibile visualizzare il mese corrente, cambiare mese, selezionare un giorno, consultare sessioni, esami e obiettivi e visualizzare l'agenda giornaliera in base allo spazio disponibile.



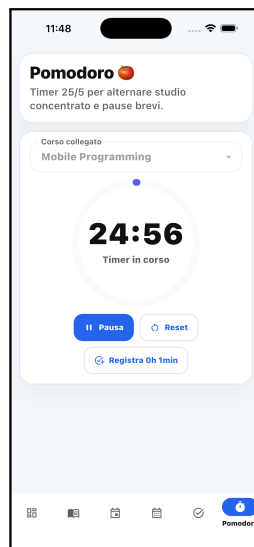
4.6.5 Obiettivi

La schermata Obiettivi gestisce le attività da completare. L'utente può creare un obiettivo, cercare, filtrare per priorità e stato, marcare un elemento come completato, modificare o eliminare un obiettivo.



4.6.6 Pomodoro

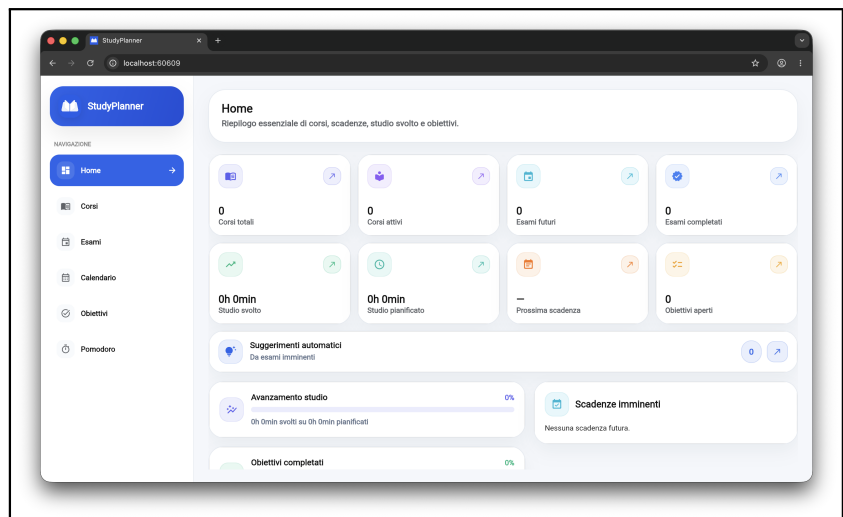
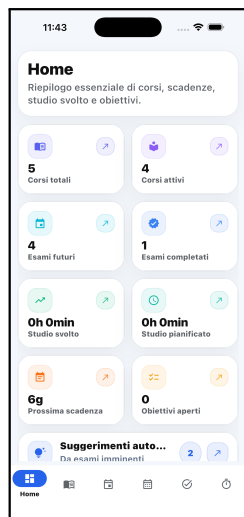
La schermata Pomodoro offre un timer integrato per lo studio concentrato. Il ciclo prevede 25 minuti di studio e 5 minuti di pausa, con pulsanti di avvio/pausa e reset.



4.7 Layout responsive e adattamento mobile

L'app è stata progettata per risultare leggibile sia su schermi ampi sia su dispositivi mobili. L'obiettivo dell'adattamento responsive è mantenere la stessa organizzazione funzionale dell'app, modificando però la disposizione degli elementi in base allo spazio disponibile.

Su schermi larghi i contenuti possono essere distribuiti in modo più ampio, mentre su smartphone vengono privilegiati layout verticali, componenti compatti e una navigazione più adatta all'uso mobile. Questa scelta consente di presentare l'app sia in ambiente browser sia su emulatore, mantenendo coerenza grafica e usabilità.



4.8 Iconografia

La tabella seguente riassume le principali icone utilizzate nell'app e il loro significato.

Sezione	Icona	Icona Flutter	Significato
Logo/App		<code>Icons.auto_stories_rounded</code>	Logo dell'applicazione.
Home		<code>Icons.dashboard_rounded</code>	Riepilogo centrale dei dati.
Corsi		<code>Icons.menu_book_rounded</code>	Insegnamenti e materiali di studio.
Esami		<code>Icons.school_rounded</code>	Appelli, prove, consegne e risultati.
Calendario		<code>Icons.calendar_month_rounded</code>	Pianificazione mensile.
Obiettivi		<code>Icons.task_alt_rounded</code>	Attività e completamento.
Pomodoro		<code>Icons.timer_rounded</code>	Timer di studio concentrato.

5 Scelte tecnologiche

5.1 Framework utilizzato

Il framework scelto è **Flutter**, con linguaggio **Dart**. Flutter permette di sviluppare un'app multi-piattaforma con componenti Material Design, layout responsivo e possibilità di esecuzione su mobile, desktop e web, questo grazie anche alla libreria Skia usata dal motore grafico del framework capace di renderizzare le schermate e trattare tutti gli elementi della UI come pixel.

Tecnologia	Motivazione
Flutter	Framework mobile cross-platform, adatto a emulatore, dispositivo fisico e browser.
Dart	Linguaggio integrato in Flutter, adatto a modelli, UI dichiarativa e asincronia.
Material 3	Componenti moderni, coerenti e facilmente adattabili.
shared_preferences	Persistenza locale semplice, sufficiente per dati manuali e fittizi.
ChangeNotifier	Gestione dello stato leggera, senza introdurre librerie complesse.
InheritedNotifier	Accesso allo stato globale da tutte le schermate.

5.2 Librerie principali

Il file `pubspec.yaml` contiene una sola dipendenza esterna significativa:

Listing 1: Dipendenze principali in `pubspec.yaml`

```
dependencies:
  flutter:
    sdk: flutter
  shared_preferences: ^2.2.3
```

La scelta di limitare le dipendenze rende il progetto più semplice da eseguire, presentare e mantenere.

5.3 Modalità di gestione della persistenza

Per la persistenza locale è stato scelto `shared_preferences`, una soluzione semplice e coerente con le esigenze del progetto. La persistenza non richiede un backend, API remote o un database esterno; per questo motivo è stata adottata una memorizzazione locale sufficiente per dati inseriti manualmente o fittizi.

La scelta consiste nel salvare lo stato dell'app in formato JSON, mantenendo i dati sul dispositivo o nell'ambiente di esecuzione. Questa soluzione riduce la complessità tecnica, facilita la demo e consente di recuperare le informazioni principali dopo il riavvio dell'applicazione.

6 Implementazione

6.1 Organizzazione del codice

Il codice principale è contenuto in `lib/main.dart`. Per una consegna didattica questa scelta rende il progetto immediatamente consultabile. A livello logico, il file è comunque organizzato in blocchi:

- avvio app e tema globale;

- `StudyScope` e `StudyStore`;
- modelli dati ed enumerazioni;
- shell di navigazione;
- componenti condivisi;
- schermate principali;
- dialog di inserimento/modifica.

In una futura evoluzione professionale, la stessa struttura potrebbe essere separata in cartelle: `models/`, `store/`, `screens/`, `widgets/`, `dialogs/`.

6.2 Gestione dello stato

Lo stato globale è gestito dalla classe `StudyStore`, che estende `ChangeNotifier`. Essa contiene le liste principali:

- `courses`;
- `exams`;
- `sessions`;
- `tasks`.

Ogni metodo di inserimento, modifica o eliminazione aggiorna la lista corrispondente, salva i dati localmente e notifica la UI.

Listing 2: Schema logico delle operazioni CRUD nello store

```
Future<void> upsertCourse(Course course) async {
  final index = courses.indexWhere((e) => e.id == course.id);

  if (index >= 0) {
    courses[index] = course;
  } else {
    courses.add(course);
  }

  await save();
  notifyListeners();
}
```

6.3 Gestione della navigazione

La navigazione è centralizzata in `ShellScreen`. La schermata selezionata viene controllata tramite un indice, che determina quale pagina mostrare tra Home, Corsi, Esami, Calendario, Obiettivi e Pomodoro.

Dal punto di vista tecnico, l'interfaccia usa `LayoutBuilder` per verificare la larghezza disponibile e adattare la navigazione:

- su schermi larghi viene costruita una struttura con `Row`, sidebar laterale e contenuto principale;

- su schermi mobili viene usato uno `Scaffold` con `NavigationBar` inferiore;
- il cambio pagina avviene aggiornando l'indice selezionato e mostrando il widget corrispondente.

Listing 3: Logica responsiva della navigazione

```
return LayoutBuilder(builder: (context, constraints) {
  if (constraints.maxWidth >= 920) {
    return Scaffold(
      body: Row(
        children: [
          _Sidebar(selected: selected, items: items, onSelected: onSelected),
          Expanded(child: pages[selected]),
        ],
      ),
    );
  }

  return Scaffold(
    body: pages[selected],
    bottomNavigationBar: NavigationBar(
      selectedIndex: selected,
      destinations: items.map(...).toList(),
      onDestinationSelected: onSelected,
    ),
  );
});
```

6.4 Passaggio dati tra schermate

Il passaggio dati non avviene duplicando informazioni tra schermate, ma tramite lo stato condiviso. Le schermate recuperano `StudyStore` con:

```
StudyScope.of(context)
```

Quando l'utente modifica un corso, un esame, una sessione o un obiettivo, il dialog riceve l'oggetto esistente oppure una data iniziale. Alla conferma viene chiamato un metodo `upsert` dello store. La dashboard e le altre sezioni si aggiornano automaticamente grazie a `notifyListeners()`.

6.5 Gestione dei dati persistenti

Dal punto di vista implementativo, ogni modello dati espone metodi `toJson()` e `fromJson()`, utilizzati per trasformare gli oggetti Dart in strutture serializzabili e per ricostruirli al caricamento dell'app.

Lo `StudyStore` raccoglie le liste principali in un'unica mappa, la converte in stringa JSON e la salva tramite `shared_preferences`. Il salvataggio viene richiamato dopo le operazioni di inserimento, modifica ed eliminazione, in modo che lo stato dell'app resti coerente anche dopo la chiusura.

Listing 4: Schema di salvataggio locale

```
await prefs.setString(storageKey, jsonEncode({
  'courses': courses.map((e) => e.toJson()).toList(),
  'exams': exams.map((e) => e.toJson()).toList(),
  'sessions': sessions.map((e) => e.toJson()).toList(),
  'tasks': tasks.map((e) => e.toJson()).toList(),
}));
```

6.6 Componenti e widget principali

Componente	Ruolo
PageFrame	Layout comune delle schermate, con header, titolo, sottotitolo e azioni.
_PageHeader	Intestazione grafica con gradiente e icona.
AppCard	Card riutilizzabile per contenuti e sezioni.
StatCard	Card sintetica per statistiche numeriche.
_Pill	Badge colorato per stato, priorità, CFU e tempi.
CourseDialog	Inserimento e modifica dei corsi.
ExamDialog	Inserimento e modifica di esami/scadenze.
SessionDialog	Inserimento e modifica di sessioni di studio.
TaskDialog	Inserimento e modifica di obiettivi.

6.7 Parti significative o complesse

Le parti più significative dell'implementazione riguardano il calendario mensile, i suggerimenti automatici e il timer Pomodoro.

6.7.1 Calendario mensile

Il calendario costruisce dinamicamente i giorni del mese, includendo anche i giorni di riempimento della settimana iniziale e finale. Per ogni giorno vengono cercate sessioni, esami e obiettivi associati.

Listing 5: Generazione dei giorni da mostrare nel calendario

```
List<DateTime> calendarDaysForMonth(DateTime month) {
  final first = DateTime(month.year, month.month, 1);
  final last = DateTime(month.year, month.month + 1, 0);
  final start = first.subtract(Duration(days: first.weekday -
    1));
  final end = last.add(Duration(days: 7 - last.weekday));

  return List.generate(
```

```

        end.difference(start).inDays + 1,
        (index) => DateTime(start.year, start.month, start.day +
            index),
    );
}

```

6.7.2 Suggerimenti automatici

La dashboard analizza gli esami futuri entro 21 giorni e propone una sessione di studio consigliata. La durata suggerita cambia in base alla vicinanza della scadenza.

Listing 6: Logica semplificata dei suggerimenti automatici

```

final upcoming = exams
  .where((e) => e.status == ItemStatus.future &&
    e.date.difference(now).inDays <= 21)
  .toList()
  ..sort((a, b) => a.date.compareTo(b.date));

final recommended = days <= 7 ? 120 : 75;

```

6.7.3 Pomodoro

Il Pomodoro usa un `Timer.periodic`. Quando il conteggio arriva a zero, l'app alterna automaticamente fase di studio e pausa.

Listing 7: Cambio automatico tra studio e pausa

```

timer = Timer.periodic(const Duration(seconds: 1), (_) {
  if (remaining <= 1) {
    setState(() {
      pause = !pause;
      remaining = pause ? pauseSeconds : workSeconds;
    });
  } else {
    setState(() => remaining--);
  }
});

```

7 Istruzioni per l'esecuzione

7.1 Prerequisiti

Per eseguire l'app sono necessari:

- Flutter SDK installato;
- Dart SDK incluso in Flutter;
- Android Studio o Visual Studio Code;

- un emulatore Android/iOS, un dispositivo fisico oppure Chrome per la versione browser.

7.2 Comandi di avvio

Dalla cartella del progetto:

Listing 8: Avvio del progetto Flutter

```
cd study_planner_app
flutter pub get
flutter run
```

Per avviare direttamente su browser Chrome:

Listing 9: Avvio su browser Chrome

```
cd study_planner_app
flutter pub get
flutter run -d chrome
```

Se la struttura di piattaforma non fosse presente, è possibile rigenerarla con:

Listing 10: Rigenerazione struttura Flutter

```
cd study_planner_app
flutter create .
flutter pub get
flutter run
```

8 Conclusioni

Con questo progetto abbiamo cercato di realizzare un'app semplice ma completa per aiutare uno studente universitario a organizzare meglio lo studio. StudyPlanner permette di gestire corsi, esami, scadenze, sessioni di studio e obiettivi personali, raccogliendo in un unico ambiente informazioni che normalmente potrebbero essere sparse tra appunti, calendari e liste separate.

Durante lo sviluppo abbiamo dato importanza soprattutto alla chiarezza dell'interfaccia, alla navigazione tra le schermate e alla coerenza dei dati. La scelta di Flutter ci ha permesso di ottenere un'app utilizzabile sia su mobile sia su browser, mantenendo una struttura responsive e adatta alla demo. Anche la persistenza locale tramite JSON e `shared_preferences` è risultata sufficiente per gli obiettivi della traccia, senza dover introdurre un backend esterno.